

Midnight ハンズオンワークショップ

ゼロ知識証明で作る、後出し不可能なジャンケンdApp

2026-09 · midnight-rps-sample-app

Who are you?



[@haruki_web3](https://twitter.com/haruki_web3)

名前: Haruki (AWS Community Builder)

Web3エンジニアコミュニティを運営してます!
ETH Tokyoなどのハッカソンに出場してます!



Xアカウント



本日のゴールとアジェンダ

 Midnight上でZK commit-revealを使ったdAppを、自分の手で動かす

1. Midnightの概要・アーキテクチャ
2. ゼロ知識証明(ZK)の基礎
3. MidnightでdAppを作るための4要素
4. 今日作るもの：Midnight RPS dApp
5. ハンズオン

01 Midnightの概要・アーキテクチャ

プライバシーを第一級市民として扱うブロックチェーン

Midnightとは

- プライバシー特化のパブリックブロックチェーン
- Cardanoエコシステムのパートナーチェーンとして稼働
- スマートコントラクトの入力・ロジックを秘匿しつつ、結果は誰でも検証可能
- ネイティブ言語 **Compact** でZK回路を記述する
- テストネット：Preprod / Preview、ネイティブトークン tNight

Cardano Partner Chain

ZK-native

Compact

なぜプライバシーが必要か

一般的なパブリックチェーン

- 残高・取引履歴が誰でも閲覧可能
- コントラクトへの入力値も丸見え
- 企業利用・金融・ゲームでは致命的

Midnightの答え

- 何を公開し何を秘匿するか設計できる
- ロジックの正しさはZK証明で保証
- 「見せずに証明する」を標準機能に

Midnightのアプローチ：選択的開示

- コントラクト開発者が**公開する情報**を明示的に選ぶ
- 秘匿したいデータは **witness** としてローカルに保持
- オンチェーンには**ハッシュ値や証明**のみが記録される
- 本ワークショップの例：ジャンケンの手は**リビールするまで誰にも見えない**

💡 「全部隠す」でも「全部見せる」でもなく、必要な粒度で公開範囲を選べる

全体アーキテクチャ



Node



Indexer



Proof Server



Lace Wallet



dApp

- **Node** — トランザクションの検証・ブロック生成
- **Indexer** — 台帳データを検索しやすい形でdAppに提供
- **Proof Server** — ZK証明の生成を担うオフチェーン計算機
- **Lace Wallet** — 秘密鍵管理・署名・shieldedアドレス
- **dApp (CLI/フロントエンド)** — ユーザー操作の入口

shielded / unshielded という考え方

tNight (shielded token)

- Midnightのネイティブトークン
- 保有量・移転を秘匿可能
- テストネットでは **faucet** から取得

DUST

- トランザクション実行に必要なガスの資源
- tNight保有から時間経過で生成
- 証明生成・TX手数料に消費される

Compact言語のポジション

- 回路 (circuit) と 台帳状態 (ledger) を1つのDSLで記述
- `compactc` コンパイラが ZK鍵一式 と TypeScript型 を生成
- Solidityとの違い：秘密の入力 (witness) を第一級で扱う構文がある
- 本リポジトリ： `pkgs/contract/src/rps.compact`

```
export ledger p1_commit: Bytes<32>;

circuit commit(move: Field, salt: Field): [] {
  // ZK回路として証明可能な形でロジックを記述
}
```

02 ゼロ知識証明 (ZK) の基礎

「見せずに証明する」を可能にする暗号技術

ゼロ知識証明とは

🔑 秘密の情報を明かさずに、「その情報を知っている」 ことだけを証明する技術

- 証明者 (Prover) は答えそのものを見せない
- 検証者 (Verifier) は計算が正しいことだけを確認できる
- 数学的に、偽の証明はほぼ100%の確率で失敗する
- Midnightではこれを `compactc` が生成するZK回路が担う

なぜdAppにZKが必要か

- ブロックチェーンの計算は本来**全ノードが再実行して検証**する
- 秘密の入力（自分の手・残高など）を使う処理は、そのままでは**公開が必須**になる
- ZK証明があれば：**秘密の入力で計算 → 結果の正しさだけを公開**
- 本ワークショップの適用先：「**自分の手**」を明かさずに勝敗を検証する

コミット・リビール方式 + ZK

- ① **Commit** — 手とsaltをハッシュ化しon-chainに送信（ZK証明つき）
- ② **待機** — 相手もコミットするまで、手は誰にも分からない
- ③ **Reveal** — 手とsaltを公開し、ハッシュが一致することを検証
- ④ **Settle** — コントラクトが勝敗を判定し記録

証明生成の流れ

- `witness` — 秘密値（手・salt）をローカルから回路に渡す
- `circuit` — Compactで書かれたZKロジック
- Proof Serverが **証明（proof）** を生成
- ノードが **検証（verify）** しTXを承認

`witness` → `circuit` → `proof` → `verify`

`rps-witnesses.ts` が `cli` / `app` 双方に秘密値を供給する

03 MidnightでdAppを作るための4要素

フロントエンド・ウォレット・スマートコントラクト・Proof Server

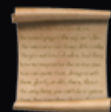
dApp構成要素の全体像



① フロントエンド



② ウォレット



③ コントラクト



④ Proof Server

この4つが揃って初めてMidnight dAppは動く。次のスライドから1つずつ見ていく

① フロントエンド：Providerチェーン

- **Wallet Provider** → 署名・アドレス取得
- **Midnight Provider** → コントラクト呼び出しの基盤
- **ZK Config Provider** → 証明に必要な鍵・回路を取得
- **Proof Provider** → Proof Serverへ証明生成を依頼
- **Private / Public Data Provider** → 秘密状態（LevelDB）／台帳状態を管理

6つのProviderを直列に組み合わせて、初めてコントラクトを呼び出せる

① フロントエンド：本リポジトリでの実例

pkgs/app

- React + Vite のブラウザdApp
- **Lace Wallet**接続で署名
- 2ブラウザで対戦できる

pkgs/cli

- Node.jsのヘッドレスCLI
- **HDウォレット**を内蔵し署名
- ブラウザ不要で対戦・検証が可能

同じコントラクトに対する**2つの独立したフロントエンド**

② ウォレット (Lace Wallet)

- ブラウザ拡張機能として動作し、`@midnight-ntwrk/dapp-connector-api` 経由で接続
- **shieldedアドレス**を管理し、TXへの署名を担当
- dApp側は `connect(networkId)` でネットワーク一致を要求する
- 対応ネットワーク：**Preprod** / **Preview**（ワークショップ内で切り替え可能）

`pkgs/cli` はLaceを使わず、内蔵HDウォレットで同等の役割を代替する

③ スマートコントラクト：ビルドフロー

- ① `rps.compact` を記述 — ledger/circuit/witnessを定義
- ② `compactc` でコンパイル — ZK鍵 (zkir/keys) + TypeScript型を生成
- ③ 生成物を `managed/rps` として `cli` ・ `app` 双方にコピー
- ④ ネットワーク非依存 — 一度コンパイルすれば全ネットワークで共用

③ コントラクト：3つの構成要素

要素	役割	本リポジトリの例
ledger	公開状態（誰でも読める）	<code>p1_commit</code> , <code>game_over</code> , <code>result</code>
circuit	ZK証明可能なロジック	<code>commit()</code> , <code>reveal()</code>
witness	秘密値の受け渡し	<code>rps-witnesses.ts</code> （手・salt）

ledgerは公開、witnessは秘密、circuitがその橋渡しを証明する

④ Proof Server — 他のチェーンにはない要素

	Ethereum / Solana	Midnight
計算の実行場所	オンチェーンで直接実行	オフチェーンのProof Serverで証明生成
ノードの仕事	計算を再実行して検証	証明を検証するだけ
秘密情報の扱い	扱えない（全て公開）	witnessとして秘匿したまま計算
dApp構成要素	フロントエンド／ウォレット／コントラクト	上記 + Proof Server

Proof ServerはMidnight特有の第4のインフラ要素

④ Proof Server の役割と配置

- ZK証明生成は **CPU負荷の高い処理** → オフチェーンに切り出す設計
- ローカル開発では **Docker** で起動する (`docker compose -f proof-server.yml up`)
- `cli` / `app` 双方が同じProof Serverを利用する
- ネットワークを切り替えても Proof Server の**再起動は不要** (ZK鍵はネットワーク非依存)

④ Proof Server : ハンズオンでの注意点

- ポート **6300** に固定でバインドされる
 - `isProofServerRunning()` が既存プロセスを検知し**二重起動を回避**
 - 2回目以降は `bun cli preprod-ps` / `preview-ps` を使うと自動判定される
-
- Devcontainer利用時は `proof-server.devcontainer.yml` を使うこと
 - 通常のcompose起動だとネットワーク分離により `ECONNREFUSED` になる

4要素のまとめ

フロントエンドが呼び出し

→ ウォレットが署名

→ コントラクトがロジックを定義

→ **Proof Server**が証明を生成

→ ノードが検証しTX確定

Frontend

Wallet

Contract

Proof Server

この一連の流れが、これから作るRPS dAppでそのまま体験できる

04 ワークショップで作るもの

Midnight RPS dApp — ZK手役かくしジャンケン

今日作るもの：ZK手役かくしジャンケン

- グー・チョキ・パーを **2人のプレイヤー** で対戦
- 手は **コミット・リビール方式** で秘匿
- 後出し・覗き見が数学的に不可能なフェアなゲーム
- デモ動画：youtu.be/jmUyDCOBVCY

今日学んだZKの概念が、そのまま1本のゲームとして動く

ゲームフロー

- ① **Commit Phase** — 手+saltをハッシュ化してon-chain送信、ZK証明を生成
- ② **(待機)** — 両者がコミットするまで相手の手は不明
- ③ **Reveal Phase** — 両者が手とsaltを公開し、ハッシュ一致を検証
- ④ **Settlement** — `player1_wins` / `player2_wins` / `draw` を判定し記録

本リポジトリのパッケージ構成

Package	役割
<code>pkgs/contract</code>	Compactコントラクト + witness + Vitestテスト
<code>pkgs/shared</code>	RPS型・ネットワーク設定・通貨定数（cli/app共通）
<code>pkgs/cli</code>	HDウォレット内蔵のヘッドレスCLI
<code>pkgs/app</code>	Lace Wallet接続のReact + Viteブラウザアプリ

`cli` と `app` は**同じコントラクト**に対する独立した2つのフロントエンド

なぜ「後出し」できないのか

- コミット時に送信するのは **手+saltのハッシュ値** のみ
- ハッシュは**一方向関数** — 逆算して元の手を求めることは事実上不可能
- リビール時に手を変えると、**ハッシュが一致せず検証に失敗**する
- コントラクトが**数学的に不正を検出**するため、信頼できる第三者は不要

「約束を破ったら即バレル」を暗号学的に保証する仕組み

今日のハンズオンの流れ

- ① 環境構築 — `bun install` / `compact update 0.30.0`
- ② コントラクトビルド — `bun contract compact` → `bun run build`
- ③ Proof Server起動 — Docker Composeでローカル起動
- ④ CLIで対戦 — `bun cli preprod-ps` でデプロイ・commit・reveal
- ⑤ (余裕があれば) ブラウザ対戦 — 2つのLaceウォレットでリアルタイム対戦

Thank you